CS3230 : Design and Analysis of Algorithms (Spring 2014)
Tutorial Set #9
[For discussion during Week 11]

OUT: 27-Mar-2014 **Tutorials:** Mon & Thu, 31 Mar, 03 Apr 2014

Prepare your answers to all the D-Problems in every tutorial set.

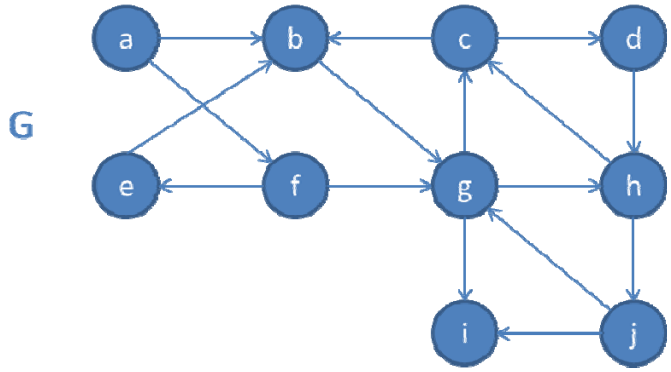
When preparing to present your answers,

- Think of a CLEAR EXPLANATION
- Illustrate with a good worked example;
- Describe the main ideas,
- Can you sketch why the solution works;
- Give analysis of running time, if appropriate
- Can you think of other (perhaps simpler) solutions?

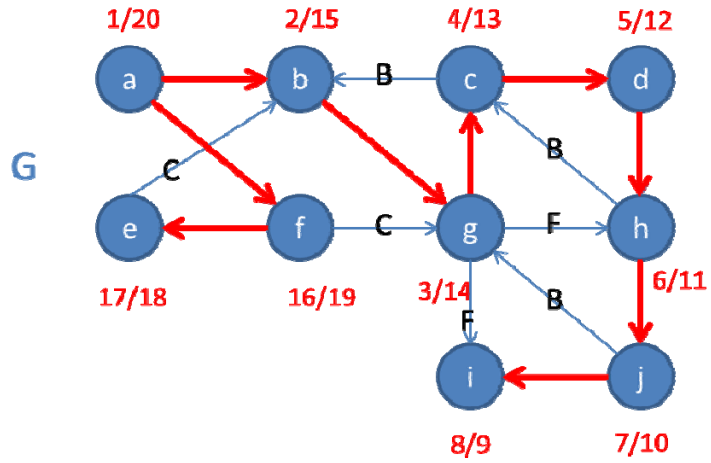
In CS3230, you learn to develop high-level abstractions when describing algorithms. Try not to speak in ML/AL (machine/assembly language) or "for (j=0; j<n; j++) do". Instead give names to your sets (of objects or things or data structures), talk about depth-first search, binary search, traverse the graph, sort the set, use a priority queue, etc. You are no longer in CS1010, CS1020, CS2010 or CS2020. Speak with greater sophistication, and at a higher level of abstraction.

***Routine Practice Problems** -- do not turn these in -- but make sure you know how to do them.*

R1. (Lecture 09 review, 4 types of edges of DFS tree)
Consider the following graph G. Can you generate the DFS tree? Then, can you write down the edge types of the edges? (The 4 edge types are: tree edge, back edge, forward edge and cross edge.)



Solution: C is cross edge, B is back edge, F is forward edge and the remaining edges are tree edges.



Discussion-Problems -- Solve these problems and present them in tutorial class.

D1. (Lecture 10 review, Recursion versus DP)

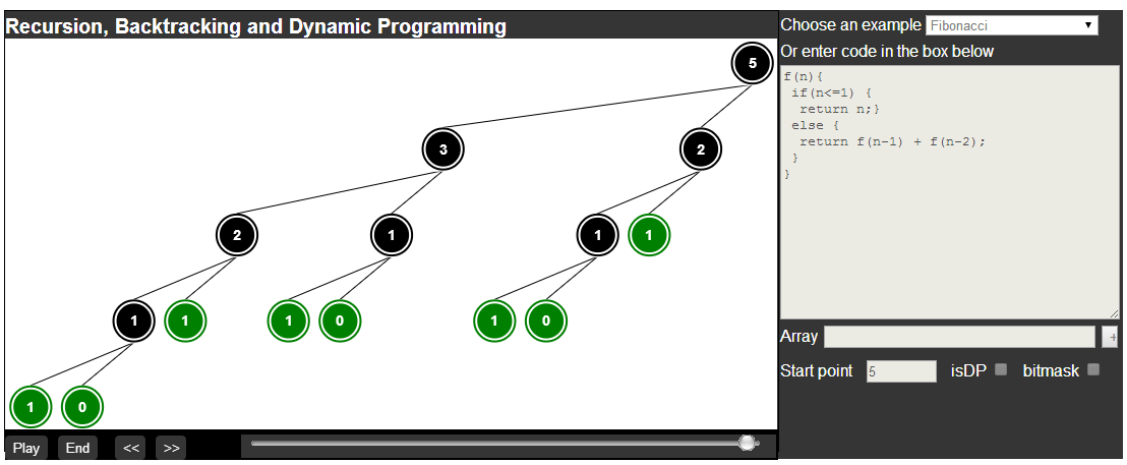
a. Consider the following recursive formula, can you draw the subproblem DAG for V(5)? What is the running time if we compute V(n) using recursion or bottom-up DP or top-down DP (memorization)? What is the name of this classic recursive formula?

Base case: V(0)=0, V(1)=1

Recursive case: V(n) = V(n-1) + V(n-2)

Solution:

This is a Fibonacci sequence, the subproblem DAG is shown below. (http://www.comp.nus.edu.sg/~stevenha/visualization/rectree.html)



Recursion: super-polynomial time (exponential time), loose bound = $O(2^n)$ as for each number n, we have two subproblems: (n-1) and (n-2). A tighter bound involves "Binet formula", or approximately $O(\sim 1.6^n)$.

Bottom-up DP: O(n) time.

Top-down DP (memoization): O(n) time.

b. Consider the following recursive formula, what is the running time if we compute W(1, n) using recursion or bottom-up DP or top-down DP (memoization)?

Base case: W(a, a) = 1 for all a

Recursive case: $W(a, b) = W(a, \lfloor (a+b)/2 \rfloor) + W(\lfloor (a+b)/2 \rfloor + 1, b) + (b-a+1)$ if a<b

Solution:

For this recurrence, if we just simulate the recursion as it is, we just need: O(n) time.
Master theorem: $T(n) = 2*T(n/2) + \Theta(1)$ is case 1, $\Theta(n)$.
Bottom-up DP: $O(n^2)$ time, as there are n^2 cells to fill in.
Top-down DP (memoization): Initializing the n^2 cells of DP table takes $O(n^2)$ time, but the running time is just O(n), essentially emulating standard recursion.

D2. (Lecture 10 review, CUT-ROD)

Given the price for a rod of various length stored in array $p = \{p_1, p_2, p_3, \ldots, p_n\}$ and a starting rod of length n, what is the output of CUT-ROD if:

a). $p_1 > (\sum p - p_1)$, note that $\sum p$ is the sum of all prices in array p
b). $p_i = p_j, \forall i, j \in [1..n]$, i.e. all length has the same price
c). $p_i < p_{i+1}, \forall i \in [1..n-1]$, i.e. the prices are sorted in ascending order

Solution:

a). We simply the rod to n copies of p_1 to get profit of $n*p_1$.
b). Same answer as a.
c). We cannot generalize anything from this test data structure.

For example, all these test cases have $p_i < p_{i+1}, \forall i \in [1..n-1]$:
1. p = {1,11,12,13} and n = 4, the answer is 11+11 = 22 (cut to two rods of length 2).
2. p = {5,10,18,21} and n = 4, the answer is 5+18 = 23 (cut to two rods of length 1 and 3).
3. p = {1,10,11,21} and n = 4, the answer is 21 (do not cut).

The output will be the output after running the DP solution of CUT-ROD.

D3. (Lecture 10 review, 0-1 Knapsack)

Suppose v[1..4] = (3, 4, 7, 2), w[1..5] = (2, 1, 3, 1) and W=5. Please illustrate the execution of the Bottom-Up Dynamic Programming. Follow up: Do you observe that you only need the values at certain row i-1 to process the current row i? Use this observation to suggest an improvement to the bottom-up DP algorithm for 0-1 Knapsack that only uses O(W) space—instead of O(nW) space—although it still runs in O(nW),

Solution:

The answer is the weight of item {2, 3, 4} = w[2]+w[3]+w[4]=5. Value = v[2]+v[3]+v[4]=13.
Recurrence: m[i, j] = max { m[i-1, j], m[i-1, j-w[i]] + v[i], if j-w[i] ≥ 0 }.
Base case: m[0, j] = 0 for all j in [0..W] and m[i, 0] = 0 for all i in [1..n].

W→	0	1	2	3	4	5
0	0	0	0	0	0	0
item = 1	0	0	3	3	3	3
item = 2	0	4	4	7	7	7
item = 3	0	4	4	7	11	11
item = 4	0	4	6	7	11	13

Improvement: Just prepare two rows and two row pointers: prev and cur.
(PS: TA can also discuss the slightly inferior solution that maintains row 0 as "previous" row and row 1 as "current row", i.e. we copy the current row to previous row at each step).

Initially, we set prev = row 0, cur = row 1, and only row prev = 0 is initialized with all zeroes and only column 0 of row cur is initialized with a single zero.

	0	**1**	**2**	**3**	**4**	**5**
prev = 0	0	0	0	0	0	0
cur = 1	0	0	3	3	3	3

Then, we set prev = 1-0 = 1, and cur = 1-1 = 0 and we proceed:

	0	**1**	**2**	**3**	**4**	**5**
cur = 0	0	4	4	7	7	7
prev = 1	0	0	3	3	3	3

Then, we set prev = 1-1 = 0, and cur = 1-0 = 1 and we proceed again:

	0	**1**	**2**	**3**	**4**	**5**
prev = 0	0	4	4	7	7	7
cur = 1	0	4	4	7	11	11

Then, we set prev = 1-0 = 1, and cur = 1-1 = 0 and we proceed again for the final time:

	0	**1**	**2**	**3**	**4**	**5**
cur = 0	0	4	6	7	11	13
prev = 1	0	4	4	7	11	11

The answer is stored in row cur = 0, last column 5, i.e. 13.
Alternative arrangement is to use row % 2 (which will yield 0 or 1) to determine which row is the previous row and which row is the current row.

D4. (Lecture 10 review, TSP, Exercise 3.5.2.6* of CP3 book)
In Lecture 10, you are given this recurrence relation **tsp(pos, bitmask)** where bitmask is a compact representation of a set of **N** items using the binary representation of integer:

Base case (if every cities have been visited):
tsp(pos, 2^N-1) = dist[pos][0]
Recursive case, try visiting unvisited cities one by one:
tsp(pos, bitmask) =
min(dist[pos][nxt] + tsp(nxt, bitmask | (1 << nxt)))
∀nxt ∈ [0..N-1], nxt != pos, bitmask & (1 << nxt) == 0

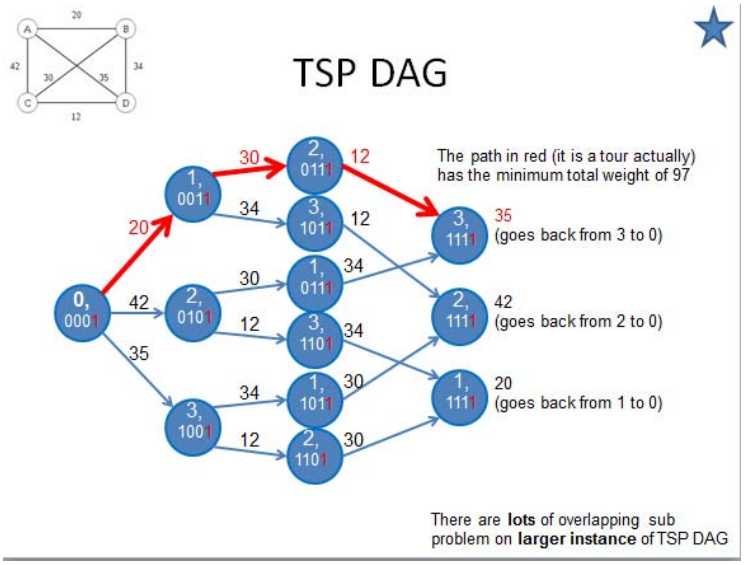
We obtain the shortest Traveling Salesman Problem (TSP) tour starting from vertex 0 (vertex A), visiting all other vertices, and returning to vertex 0 by calling **tsp(0, 1)**.

The subproblems of TSP recurrence can overlap as shown in lecture, but if we ensure that each subproblem is only computed once, we can solve TSP problem in $O(2^n * n^2)$.

However, the recurrence above is slightly inefficient. We can fix all tours to start and end at vertex 0 (vertex A). Can you modify the recurrence slightly to improve the runtime to $O(2^{n-1} * n^2)$?

Solution:

In the original recurrence, the last bit of bitmask will always be 1 as we start with tsp(0, 1). This is a waste of precious space.



By assuming that the last bit (vertex 0) in bitmask is always 1/visited anyway, we can actually "drop that last bit" and slightly alter the tsp recurrence into tsp2 as shown below:

Base case (if every cities have been visited, including vertex 0 that always 1/visited):

tsp2(pos, 2^{N-1}-1) = dist[pos][0]

Recursive case, try visiting unvisited cities one by one:

tsp2(pos, bitmask) =

min(dist[pos][nxt] + **tsp2**(nxt, bitmask | (1 << **(nxt-1)**)))

∀nxt ∈ [**1**..N-1], nxt != pos, bitmask & (1 << **(nxt-1)**) == 0

Now, we call **tsp2**(0, **0**). This way, there are only $O(2^{N-1} * N)$ vertices to consider in the subproblem DAG and we still have up to O(N) per vertex. Thus the overall time complexity is $O(2^{N-1} * N^2)$ which is more efficient.

PS: This algorithm has a name, Held and Karp's algorithm, invented back in 1962, at that time used to solve TSP of size n = 13 :O using the computer speed of that era. At that time, it was a breakthrough as $2^{12} * 13 = 53{,}248$ is much smaller compared to all 13! = 6,227,020,800 tours.

Reference: *Held, Michael; Karp, Richard Manning. (1962), "A Dynamic Programming Approach to Sequencing Problems", Journal of the Society for Industrial and Applied Mathematics 10 (1): 196–210, doi:10.1137/0110015*

Advanced Problems-Try these for challenge and fun. There is no deadline for A-problems.
Turn in your attempts DIRECTLY to Steven. Do not combine it with your HW solutions.

Attempt Exercise 3.5.2.5* of Competitive Programming 3 (page 111).
All past and present CS3233 students are excluded.
Yes, to get access to this question, you need to have CP3 book.

Hint: The name of this problem is “bounded knapsack problem”.